

1.7. Massivlarni birlashtirish, bo'lish va agregatsiyalash (Combining, Splitting and Aggregating Arrays)

Massivlarni birlashtirish kichik massivlarni bitta yirik massivga jamlashni anglatadi. Massivlarni vertikal, gorizontal yoki istalgan maxsus o'q bo'ylab birlashtirish mumkin. Bu jarayon konkatenatsiya (ulash) va steklash (gorizontal, vertikal taxlash) kabi usullarni o'z ichiga oladi.

1.7.1. NumPy massivlarini birlashtirish (Joining NumPy Arrays)

NumPy massivlarini birlashtirish bir nechta massivni bitta yirik massivga jamlashni anglatadi. Masalan, [1, 2] va [3, 4] massivlarini birlashtirish natijasida [1, 2, 3, 4] massivi hosil bo'ladi. Keling, NumPy yordamida massivlarni birlashtirishning eng keng tarqalgan usullarini ko'rib chiqamiz.

1. `numpy.concatenate()` funksiyasidan foydalanish

`numpy.concatenate()` funksiyasi ikki yoki undan ortiq massivni yangi o'lcham qo'shmasdan, mavjud o'q bo'ylab birlashtiradi. Bu usul massivlarni oddiygina ulab qo'yish uchun juda tez va samarali hisoblanadi.

Misol

```
import numpy as np

a = np.array([1, 2])
b = np.array([3, 4])

# Massivlarni birlashtirish
res = np.concatenate((a, b))

print(res)
```

[1 2 3 4]

Ushbu kod NumPy-ning `concatenate` funksiyasidan foydalanib, massivlarni uchma-uch birlashtiradi va bitta uzun [1, 2, 3, 4] ro'yxatiga aylantiradi.

Eslatma: Standart holatda (agar o'q ko'rsatilmasa) `concatenate` birinchi o'q (`axis=0`) bo'ylab ishlaydi. 1-D massivlar uchun bu ularni shunchaki yonma-yon ulab qo'yishni anglatadi.

2. `numpy.hstack()` / `numpy.vstack()` / `numpy.dstack()` funksiyalaridan foydalanish

`numpy.hstack()`, `numpy.vstack()` va `numpy.dstack()` funksiyalari `concatenate` funksiyasining qulay "o'ramlari" (wrappers) bo'lib, massivlarni gorizontaal, vertikal yoki chuqurlik bo'ylab taxlash (stacking) uchun ishlatiladi. Ular kodni yanada o'qishli va tushunarli qiladi.

Misol

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Gorizontaal taxlash
res_h = np.hstack((a, b))
print("np.hstack: ", res_h)

# Vertikal taxlash
res_v = np.vstack((a, b))
print("np.vstack: ", res_v)

# Chuqurlik bo'ylab taxlash
res_d = np.dstack((a, b))
print("np.dstack: ", res_d)

np.hstack:  [1 2 3 4 5 6]
np.vstack:  [[1 2 3]
 [4 5 6]]
np.dstack:  [[[1 4]
 [2 5]
 [3 6]]]
```

- **hstack:** massivlarni bitta uzun massivga birlashtiradi.
- **vstack:** ularni 2D massivda qatorlar ko'rinishida ustma-ust taxlaydi.
- **dstack:** ularni uchinchi o'q (chuqurlik) bo'ylab birlashtirib, 3D massiv hosil qiladi.

Izoh: `dstack` funksiyasida birinchi massivning birinchi elementi bilan ikkinchi massivning birinchi elementi bitta chuqurlik qatlamiga tushadi (masalan, `[1 4]`). Bu usul rangli tasvirlar bilan ishlashda (RGB kanallarini birlashtirishda) juda ko'p qo'llaniladi.

3. `numpy.stack()` funksiyasidan foydalanish

`numpy.stack()` massivlarni **yangi o‘q** bo‘ylab birlashtiradi va natijada o‘lchovlar sonini (dimensionality) oshiradi. Bu usul massivlarni birlashtirish bilan birga, ularni yangi o‘lchovda alohida saqlab qolishni xohlaganingizda foydali bo‘ladi.

Misol

```
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

# Massivlarni 1-o‘q bo‘ylab taxlash
res = np.stack((a, b), axis=1)

print(res)
```

```
[[1 5]
 [2 6]
 [3 7]
 [4 8]]
```

`axis=1` parametri bilan ishlatilgan `stack` funksiyasi elementlarni juftlik holatida yonma-yon qatorlarga birlashtiradi. Natijada har bir qatori har bir massivdan bittadan elementni o‘z ichiga olgan 2D massiv hosil bo‘ladi.

4. `numpy.block()` funksiyasidan foydalanish

`numpy.block()` massivlarni ichma-ich bloklardan xuddi matrisalarni kichikroq sub-massivlardan yig‘gandek quradi. U murakkab massiv maketlarini yaratish uchun moslashuvchan va kuchli vositadir.

Misol

```
b1 = np.array([[1, 1], [1, 1]])
b2 = np.array([[2, 2, 2], [2, 2, 2]])
b3 = np.array([[3, 3], [3, 3], [3, 3]])
b4 = np.array([[4, 4, 4], [4, 4, 4], [4, 4, 4]])

# Bloklarni joylashtirish
res = np.block([
    [b1, b2],
    [b3, b4]
])

print(res)
```

```
[[1 1 2 2 2]
 [1 1 2 2 2]
 [3 3 4 4 4]
 [3 3 4 4 4]
 [3 3 4 4 4]]
```

To'rtta kichik massiv bloklar ko'rinishida joylashtirilgan: `b1` va `b2` yuqorida yonma-yon, `b3` va `b4` esa ularning ostida yonma-yon. `np.block()` ushbu tartibni saqlagan holda ularni bitta yirik massivga birlashtiradi.

Eslatma: `np.block` ishlatayotganda bloklar o'lchami mos kelishi kerak. Masalan, bir qatorda turgan bloklarning balandligi (qatorlar soni) bir xil bo'lishi shart.

1.7.2. 1-D va 2-D massivlarni birlashtirish (Combining 1-D and 2-D Arrays)

Ba'zan 1-D massivni 2-D massiv bilan birlashtirish va ularning elementlari ustida birgalikda takrorlanish (iteratsiya) talab qilinadi. NumPy `numpy.nditer()` funksiyasi yordamida bu vazifani osonlashtiradi.

Misol: Ushbu oddiy kod `nditer()` yordamida 1-D va 2-D massivlarni qanday birgalikda qo'llash mumkinligini ko'rsatadi.

```
num1 = np.array([1, 2])
num2 = np.array([[10, 20],
                 [30, 40]])

for a, b in np.nditer([num1, num2]):
    print(a, ":", b)
```

```
1 : 10
2 : 20
1 : 30
2 : 40
```

Bu yerda 1-D massiv elementlari 2-D massiv bilan juftlik hosil qilish uchun qatorlar bo'yicha takrorlanadi.

`nditer` funksiyasi

NumPy-dagi `nditer` funksiyasi — bu massivlar turli shaklga (shape) ega bo'lsa ham, ularni samarali aylanib chiqish (loop) imkonini beruvchi iteratoridir. Unga ikkita massiv (masalan, 1-D va 2-D) berilsa, u elementlarni o'zaro juftlaydi va ularni birgalikda qayta ishlashga sharoit yaratadi.

Sintaksis `np.nditer([massiv1, massiv2])`

Misol

Ushbu misolda biz 5 ta elementdan iborat 1-D massivni 2x5 o'lchamli 2-D massiv bilan birlashtiramiz va ularni birgalikda ko'rsatamiz.

```
num1 = np.arange(5)
print("1D massiv:")
print(num1)

num2 = np.arange(10).reshape(2, 5)
print("\n2D massiv:")
print(num2)

# 1-D va 2-D massivlarni birlashtirib iteratsiya qilish
for a, b in np.nditer([num1, num2]):
    print("%d:%d" % (a, b))
```

1D massiv:

[0 1 2 3 4]

2D massiv:

[[0 1 2 3 4]

[5 6 7 8 9]]

0:0

1:1

2:2

3:3

4:4

0:5

1:6

2:7

3:8

4:9

E'tibor bering, 1-D massiv 2-D massivning qatorlariga mos kelishi uchun o'z-o'zini takrorlamoqda.

2-misol: Endi uzunroq 1-D massiv va kattaroq 2-D massiv bilan sinab ko'ramiz.

```
num1 = np.arange(7)
print("1D massiv:")
print(num1)

num2 = np.arange(21).reshape(3, 7)
print("\n2D massiv:")
```

```
print(num2)
```

```
# nditer yordamida birgalikda aylanish
```

```
for a, b in np.nditer([num1, num2]):  
    print("%d:%d" % (a, b))
```

1D massiv:

```
[0 1 2 3 4 5 6]
```

2D massiv:

```
[[ 0  1  2  3  4  5  6]  
 [ 7  8  9 10 11 12 13]  
 [14 15 16 17 18 19 20]]
```

0:0

1:1

2:2

3:3

4:4

5:5

6:6

0:7

1:8

2:9

3:10

4:11

5:12

6:13

0:14

1:15

2:16

3:17

4:18

5:19

6:20

Izoh: Bu misolda 1-D massiv (7 ta element) 2-D massivning (3x7 o'lchamli) har bir qatori bilan mos keladi. `nditer` funksiyasi 1-D massivni virtual ravishda 3 marta takrorlaydi (har bir qator uchun bir marta), natijada jami 21 ta juftlik hosil bo'ladi. Bu ma'lumotlarni tahlil qilishda bitta o'lchovli koeffitsientlarni ko'p o'lchamli ma'lumotlar to'plamiga nisbatan qo'llashda juda qo'l keladi.

3-misol: Bu yerda 1-D massiv atigi 2 ta elementdan iborat, ammo 2-D massiv ko'plab qatorlarga ega.

```

num1 = np.arange(2)
print("1D massiv:")
print(num1)

num2 = np.arange(12).reshape(6, 2)
print("\n2D massiv:")
print(num2)

# nditer yordamida juftliklarni chiqarish
for a, b in np.nditer([num1, num2]):
    print("%d:%d" % (a, b))

```

```

1D massiv:
[0 1]

```

```

2D massiv:
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
0:0
1:1
0:2
1:3
0:4
1:5
0:6
1:7
0:8
1:9
0:10
1:11

```

Izoh: Ushbu misolda `num1` massivi `[0, 1]` ko'rinishida. 2-D massiv (`num2`) esa 6 ta qator va 2 ta ustundan iborat. `nditer` funksiyasi `num1` ning har bir elementini `num2` ning mos ustunidagi elementlar bilan juftlaydi. 1-D massiv 2-D massivning barcha 6 ta qatori bilan moslashish uchun jami 6 marta takrorlanadi. Bu "Broadcasting" (translyatsiya) mexanizmining yaqqol namunasidir.

1.7.3. Massivlarni bo'laklarga ajratish (Splitting Arrays in NumPy)

Massivlarni bo'lish — yirik massivni kichikroq va boshqarish oson bo'lgan sub-massivlarga ajratish jarayonidir. Bo'lish qatorlar, ustunlar yoki boshqa o'qlar bo'ylab amalga oshirilishi mumkin.

NumPy turli yo'nalishlar (qatorlar, ustunlar, chuqurlik) bo'ylab massivlarni bo'lishni osonlashtiradigan bir qancha funksiyalarni taqdim etadi.

Massivlarni bo'lishda tushunish kerak bo'lgan muhim atamalar:

- **O'q (Axis):** Massiv bo'linadigan yo'nalish (qatorlar uchun 0, ustunlar uchun 1).
- **Sub-massivlar:** Asl massivni bo'lish natijasida yaratilgan kichikroq massivlar.
- **Bo'lish metodlari:** `np.split()`, `np.hsplit()`, `np.vsplit()` va `np.array_split()` kabi funksiyalar.
- **Teng va teng bo'lmagan bo'lish:** ma'lumotlar teng qismlarga bo'linishi mumkin yoki zarurat bo'lsa, biroz teng bo'lmagan qismlarga bo'linishi mumkin (`array_split()` yordamida).

Misol: Ushbu misolda `np.array_split()` yordamida 1-D massiv uchta kichikroq qismga bo'linadi.

```
arr = np.array([1, 2, 3, 4, 5, 6])
res = np.array_split(arr, 3)

print(res)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

Izoh: `np.array_split(arr, 3)` massivni elementlarni iloji boricha teng taqsimlagan holda 3 ta sub-massivga ajratadi.

Bo'lish metodlari

NumPy massivlarni kichikroq qismlarga bo'lish uchun bir nechta o'rnatilgan funksiyalarni taqdim etadi. Ushbu metodlar 1-D, 2-D va hatto 3-D massivlarni turli o'qlar bo'ylab ajratishga yordam beradi. Keling, har bir metodni oddiy misollar, natijalar va tushuntirishlar bilan birma-bir ko'rib chiqamiz.

1. `numpy.split()`

`numpy.split()` funksiyasi massivni **teng o'lchamli** sub-massivlarga bo'lish uchun ishlatiladi. Bo'laklar soni tanlangan o'q bo'ylab massiv hajmini qoldiqsiz (aniq) bo'lishi shart. Agar teng bo'lish imkoni bo'lmasa, NumPy xatolik (`ValueError`) qaytaradi.

Misol:

```
# 0 dan 5 gacha bo'lgan sonlardan massiv yaratamiz
arr = np.arange(6)
# Massivni 2 ta teng qismga bo'lamiz
res = np.split(arr, 2)

print(res)

[array([0, 1, 2]), array([3, 4, 5])]
```

Izoh: `np.arange(6)` funksiyasi `[0, 1, 2, 3, 4, 5]` massivini yaratadi. `np.split(arr, 2)` esa uni 2 ta teng qismga ajratadi va natijada har biri 3 ta elementdan iborat ikkita sub-massiv hosil bo'ladi.

2. `numpy.array_split()`

`numpy.array_split()` funksiyasi `split()` kabi ishlaydi, lekin u **teng bo'lmagan** bo'lishga ruxsat beradi. Bu massiv hajmi bo'laklar soniga qoldiqsiz bo'linmagan hollarda juda foydali. NumPy ortiqcha elementlarni avtomatik ravishda birinchi sub-massivlar o'rtasida taqsimlaydi.

Misol:

```
# 13 ta elementdan iborat massiv yaratamiz
arr = np.arange(13)
# Massivni 4 ta qismga bo'lamiz
res = np.array_split(arr, 4)

print(res)

[array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8, 9]), array([10, 11, 12])]
```

Izoh: `np.arange(13)` 13 ta elementdan iborat massiv yaratadi. `np.array_split(arr, 4)` uni 4 ta teng bo'lmagan qismga ajratadi: birinchi sub-massiv 4 ta elementni oladi, qolgan uchta esa 3 tadan elementga ega bo'ladi.

3. `numpy.vsplit()`

`numpy.vsplit()` funksiyasi **vertikal bo'lishni** amalga oshiradi, ya'ni u matrisani qatorlar bo'yicha (0-o'q bo'ylab) ajratadi. Bu funksiya faqat 2 yoki undan ortiq o'lchamli massivlar bilan ishlaydi.

Misol:

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
# Massivni vertikal ravishda 2 qismga bo'lamiz
res = np.vsplit(arr, 2)

print(res)
```

```
[array([[1, 2, 3],
        [4, 5, 6]]), array([[ 7,  8,  9],
        [10, 11, 12]])]
```

Izoh: `vsplit(matrix, 2)` matrisani 2 ta vertikal (qatorlar bo'yicha) qismga ajratadi va har bir qism 2 tadan qatorni o'z ichiga oladi.

4. `numpy.hsplit()`

`numpy.hsplit()` **gorizontal bo'lishni** amalga oshiradi, ya'ni massivni ustunlar bo'yicha (1-o'q bo'ylab) ajratadi. Bu usul ma'lumotlar to'plamidagi turli xususiyatlar (feature) ustunlarini ajratib olishda juda qo'l keladi.

Misol:

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
# Massivni gorizontal ravishda 2 qismga bo'lamiz
res = np.hsplit(arr, 2)

print(res)
```

```
[array([[ 1,  2],
        [ 5,  6],
        [ 9, 10]]), array([[ 3,  4],
        [ 7,  8],
        [11, 12]])]
```

Izoh: `hsplit(array, 2)` massivni 2 ta teng ustunlar guruhiga ajratadi va har bir natijaviy sub-massiv 2 tadan ustunni o'z ichiga oladi.

5. `numpy.dsplitle()`

`numpy.dsplitle()` funksiyasi **3D massivlar** uchun ishlatiladi. U massivni uchinchi o'q (chuqurlik yoki 2-o'q) bo'ylab bo'ladi. Bu steklangan matrisalar, tasvirlar yoki ko'p kanalli ma'lumotlar bilan ishlashda foydalidir.

Misol:

```
# 24 ta elementdan iborat 3D massiv yaratamiz (shakli: 2, 3, 4)
arr = np.arange(24).reshape((2, 3, 4))
# Oxirgi o'q bo'ylab 2 qismga bo'lamiz
res = np.dsplit(arr, 2)

print(res)
```

```
[array([[ 0,  1],
        [ 4,  5],
        [ 8,  9]],

        [[12, 13],
         [16, 17],
         [20, 21]]), array([[ 2,  3],
        [ 6,  7],
        [10, 11]],

        [[14, 15],
         [18, 19],
         [22, 23]])]
```

Izoh: Massiv shakli (2, 3, 4) va `dsplit(..., 2)` oxirgi o'qni 2 ta teng qismga ajratadi. Har bir natija oxirgi o'lchamning yarmini (ya'ni 2 ta elementdan iborat chuqurlikni) o'z ichiga oladi.

1.7.4. Massivlarni agregatsiyalash (Aggregating Arrays)

Aggregatsiya — bu yig'indini hisoblash, o'rtacha qiymatni topish yoki maksimal/minimal qiymatlarni aniqlash kabi matematik amallarni qo'llash orqali massiv ichidagi ma'lumotlarni umumlashtirishni anglatadi.

Python-da `numpy.sum()`

`numpy.sum()` — bu massiv elementlarining yig'indisini hisoblash uchun ishlatiladigan NumPy funksiyasidir. U qiymatlarni butun massiv bo'ylab yoki ma'lum bir o'q (axis) bo'ylab jamlashi mumkin. Shuningdek, u natijaning ma'lumot turi (dtype), boshlang'ich qiymati (initial) va shaklini (keepdims) nazorat qilish imkonini beradi.

```
arr = np.array([5, 10, 15])
print(np.sum(arr))
```

Izoh: `np.sum(arr)` barcha elementlarni qo'shadi ($5 + 10 + 15$) va umumiy natijani qaytaradi.

Sintaksis:

```
numpy.sum(arr, axis=None, dtype=None, out=None, initial=0, keepdims=Fa
```

Parametrlar:

- `arr`: Yig'indisi hisoblanishi kerak bo'lgan kirish massivi.
- `axis`: Yig'indi hisoblanadigan o'q. `0` -> ustunlar bo'yicha, `1` -> qatorlar bo'yicha, `None` -> butun massiv bo'yicha.
- `dtype`: Qaytariladigan yig'indining ma'lumot turi.
- `out`: Natijani saqlash uchun mo'ljallangan chiqish massivi.
- `initial`: Yig'indining boshlang'ich qiymati.
- `keepdims`: Natijada kamaytirilgan o'qlarni o'lcham sifatida saqlab qoladi.

1-misol: 1D massiv va dtype ta'siri

Ushbu misol `numpy.sum()` funksiyasining 1D massivda qanday ishlashini va `dtype` (ma'lumot turi)ni o'zgartirish natijaga qanday ta'sir qilishini ko'rsatadi.

```
arr = np.array([20, 2, 0.2, 10, 4])

print(np.sum(arr))                # Odatiy yig'indi
print(np.sum(arr, dtype=np.uint8)) # 8-bitli butun son ko'rinishida
print(np.sum(arr, dtype=np.float32)) # 32-bitli suzuvchi nuqtali son ko'rini
```

36.2

36

36.2

Izoh:

1. `np.sum(arr)` NumPy-ning standart ma'lumot turi yordamida yig'indini hisoblaydi va o'nlik qismlarni saqlab qoladi.
2. `np.sum(arr, dtype=np.uint8)` natijani 8-bitli ishorasiz butun songa aylantiradi (bu tur faqat 0 dan 255 gacha bo'lgan sonlarni qo'llab-quvvatlaydi). Bunda o'nlik qismlar tashlab yuboriladi.

3. `np.sum(arr, dtype=np.float32)` suzuvchi nuqtali arifmetika yordamida yig'indini hisoblaydi va aniqlikni saqlaydi.

Eslatma: `uint8` kabi kichik butun son turlaridan foydalanish **overflow** (to'lib ketish) tufayli kutilmagan natijalarga olib kelishi mumkin. Bu hisoblash xatosi emas, balki tanlangan ma'lumot turining sig'imi bilan bog'liq.

2-misol: Ushbu misol 2D massivning yig'indisini hisoblashni va turli ma'lumot turlaridan foydalanish natijani qanday o'zgartirishini ko'rsatadi.

```
arr = np.array([ [14, 17, 12, 33, 44],
                 [15, 6, 27, 8, 19],
                 [23, 2, 54, 1, 4] ])

print(np.sum(arr))
print(np.sum(arr, dtype=np.uint8))
print(np.sum(arr, dtype=np.float32))
```

```
279
23
279.0
```

Izoh:

- `np.sum(arr)` umumiy yig'indi 279 ekanini ko'rsatadi.
- `dtype=np.uint8` ishlatilganda natija 23 bo'lib qoldi. Bunga sabab — `uint8` turi faqat 255 gacha bo'lgan sonlarni sig'dira oladi. Yig'indi 279 bo'lgani uchun u to'lib ketdi (**overflow**) va $279 - 256 = 23$ natijasini berdi.

3-misol: Ushbu misol 2D massivni qatorlar va ustunlar bo'ylab jamlashni hamda `keepdims=True` parametrining vazifasini namoyish etadi.

```
arr = np.array([ [14, 17, 12, 33, 44],
                 [15, 6, 27, 8, 19],
                 [23, 2, 54, 1, 4] ])

print(np.sum(arr))
print(np.sum(arr, axis=0))
print(np.sum(arr, axis=1))
print(np.sum(arr, axis=1, keepdims=True))
```

279

```
[52 25 93 42 67]
```

```
[120 75 84]
```

```
[[120]
```

```
 [ 75]
```

```
 [ 84]]
```

Izoh:

- `np.sum(arr, axis=0)` : Ustunlar bo'yicha yig'indi (pastga qarab jamlash).
- `np.sum(arr, axis=1)` : Qatorlar bo'yicha yig'indi (o'ngga qarab jamlash).
- `np.sum(arr, axis=1, keepdims=True)` : Kamaytirilgan o'lchamni saqlab qoladi va natijani ustun shaklidagi 2D massiv ko'rinishida qaytaradi. Bu keyinchalik massivlar bilan matematik amallar bajarishda (masalan, broadcasting) juda foydali.

Python-da `numpy.mean()`

`numpy.mean()` — bu sonli qiymatlarning o'rtacha qiymatini (arifmetik o'rtacha) hisoblash uchun ishlatiladigan NumPy funksiyasidir. U 1D ro'yxat/massivning o'rtachasini yoki ko'p o'lchamli massivlar uchun qatorlar va ustunlar bo'yicha o'rtacha qiymatni hisoblashi mumkin.

Misol: Kirish: `[1, 2, 3]` Natija: `2.0`

Sintaksis

NumPy-da o'rtacha qiymatni hisoblash uchun quyidagi sintaksisdan foydalanamiz: `numpy.mean(arr, axis=None, dtype=None, out=None)`

Parametrlar:

- `arr`: Sonlardan iborat kirish massivi.
- `axis`: `None` - barcha elementlarning o'rtachasi, `0` - ustunlar bo'yicha o'rtacha va `1` - qatorlar bo'yicha o'rtacha.
- `dtype` (**Ixtiyoriy**): Hisoblash jarayonida foydalaniladigan ma'lumot turi.
- `out` (**Ixtiyoriy**): Natijani saqlash uchun massiv.

Misollar

1-misol: Ushbu misol `np.mean()` yordamida 1D ro'yxatning o'rtacha qiymatini topadi.

```
arr = [20, 2, 7, 1, 34]
res = np.mean(arr)

print(res)
```

12.8

Izoh: $(20 + 2 + 7 + 1 + 34)/5 = 12.8$

2-misol: Ushbu misol `axis` parametridan foydalanib, barcha elementlarning, har bir ustunning va har bir qatorning o'rtacha qiymatini qanday hisoblashni ko'rsatadi.

```
arr = [[14, 17, 12],
        [15, 6, 27],
        [23, 2, 54]]

print(np.mean(arr))           # butun massiv bo'yicha
print(np.mean(arr, axis=0))   # ustunlar bo'yicha o'rtacha
print(np.mean(arr, axis=1))   # qatorlar bo'yicha o'rtacha
```

```
18.888888888888889
[17.33333333  8.33333333 31.          ]
[14.33333333 16.          26.33333333]
```

3-misol: Ushbu misol `out` parametridan foydalanib, qatorlar bo'yicha hisoblangan o'rtacha qiymat natijasini boshqa bir massivga saqlashni namoyish etadi.

```
arr = [[5, 10, 15],
        [3, 6, 9],
        [8, 16, 24]]

# Natijani saqlash uchun bo'sh massiv yaratamiz
res = np.zeros(3)
np.mean(arr, axis=1, out=res)

print(res)
```

```
[10.  6. 16.]
```

Izoh: `out=res` parametri qatorlar bo'yicha hisoblangan o'rtacha qiymatlarni to'g'ridan-to'g'ri `res` massivga joylashtiradi. Bu usul xotirani tejash uchun foydalidir, chunki yangi massiv yaratish o'rniga mavjudidan foydalaniladi.

NumPy-da `sum` va `mean` dan tashqari ma'lumotlarni tahlil qilishda eng ko'p ishlatiladigan boshqa agregat funksiyalar ham mavjud. Ular ma'lumotlarning tarqalishi va ekstremal qiymatlarini aniqlashda yordam beradi.

`numpy.min()` va `numpy.max()`

Ushbu funksiyalar massivdagi eng kichik va eng katta qiymatlarni topadi.

```
arr = np.array([[10, 2, 45],
                [1, 55, 12]])

print("Eng kichik qiymat:", np.min(arr)) # Butun massivda
print("Har bir ustundagi eng katta:", np.max(arr, axis=0)) # Ustunlar bo'yida
```

Eng kichik qiymat: 1

Har bir ustundagi eng katta: [10 55 45]

Izoh: Bu funksiyalar ko'pincha ma'lumotlarni normallashtirish (0 dan 1 gacha bo'lgan oraliqqa keltirish) jarayonida ishlatiladi.

`numpy.median()`

Mediana — bu tartiblangan ma'lumotlar to'plamining qoq o'rtasidagi qiymat. U o'rtacha qiymatdan (`mean`) farqli o'laroq, ekstremal "shovqinli" qiymatlarga (outliers) juda chidamli.

```
arr = np.array([1, 2, 3, 4, 1000]) # 1000 - shovqinli qiymat

print("O'rtacha (Mean):", np.mean(arr)) # 202.0 (haqiqatni aks ettirmaydi)
print("Mediana (Median):", np.median(arr)) # 3.0 (markazni aniq ko'rsatadi)
```

O'rtacha (Mean): 202.0

Mediana (Median): 3.0

Izoh: Agar ma'lumotlaringizda keskin katta yoki kichik xato qiymatlar bo'lsa, `mean` o'rniga `median` ishlatish tavsiya etiladi.

`numpy.std()` va `numpy.var()`

- **std() (Standard Deviation):** Standart og'ish — ma'lumotlarning o'rtacha qiymatdan qanchalik tarqalganini (dispersiya darajasini) ko'rsatadi.
- **var() (Variance):** Dispersiya — standart og'ishning kvadrati.

```
arr = np.array([1, 5, 10])
```

```
print("Standart og'ish:", np.std(arr))
print("Dispersiya:", np.var(arr))
```

Standart og'ish: 3.6817870057290873
Dispersiya: 13.555555555555557

Izoh: Standart og'ish qanchalik kichik bo'lsa, ma'lumotlar o'rtacha qiymat atrofida shunchalik zich joylashgan bo'ladi.

numpy.prod()

Massiv elementlarining ko'paytmasini hisoblaydi.

```
arr = np.array([1, 2, 3, 4])
print("Elementlar ko'paytmasi:", np.prod(arr)) # 1*2*3*4 = 24
```

Elementlar ko'paytmasi: 24

numpy.cumsum()

Elementlarning o'sib boruvchi (to'planib boruvchi) yig'indisini hisoblaydi.

```
arr = np.array([1, 2, 3, 4])
print("To'planuvchi yig'indi:", np.cumsum(arr))
# Natija: [1, 3, 6, 10] (1, 1+2, 1+2+3, 1+2+3+4)
```

To'planuvchi yig'indi: [1 3 6 10]

Izoh: Bu moliya sohasida (masalan, kunlik foydani umumiy hisoblab borishda) juda keng qo'llaniladi.

Umumiy jadval

Funksiya	Vazifasi
<code>np.sum()</code>	Yig'indi
<code>np.mean()</code>	o'rtacha qiymat
<code>np.median()</code>	Mediana (markaziy qiymat)
<code>np.min()</code> / <code>np.max()</code>	Minimal / Maksimal
<code>np.std()</code>	Standart og'ish (tarqalish)
<code>np.prod()</code>	Ko'paytma

Funksiya

Vazifasi

`np.cumsum()`

Ketma-ket yig'indi



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy massivlarini birlashtirishda `np.concatenate()` va `np.stack()` funksiyalarining asosiy farqi nimada?
2. `np.hstack()` va `np.vstack()` funksiyalari massivlarni qaysi yo'nalishlar bo'ylab birlashtiradi?
3. Massivlarni bo'lish uchun ishlatiladigan `np.split()` funksiyasida massivni teng bo'laklarga bo'lish imkonsiz bo'lsa, nima sodir bo'ladi?
4. Agregat funksiyalar (masalan, `np.sum()`, `np.mean()`) nima vazifani bajaradi va ular ma'lumotlar tahlilida nima uchun kerak?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `np.concatenate()` funksiyasida `axis` parametri berilmasa, u odatda qaysi o'q bo'yicha birlashtiradi?
6. `np.dstack()` funksiyasi massivlarni qaysi o'lcham (chuqurlik) bo'ylab birlashtirish uchun mo'ljallangan?
7. `np.hsplit()` va `np.vsplit()` funksiyalari 2D massivlarni qanday tartibda bo'laklarga ajratadi?
8. Agregatsiyalashda `axis=0` va `axis=1` parametrlari hisob-kitob yo'nalishiga qanday ta'sir qiladi?

3-daraja: Amaliy tahlil va statistik funksiyalar

9. `np.column_stack()` funksiyasi 1D massivlarni 2D massivga birlashtirishda qanday mantiq asosida ishlaydi?
10. `np.array_split()` funksiyasining oddiy `np.split()` dan asosiy afzalligi nimada?
11. `np.cumsum()` funksiyasi qanday natija qaytaradi va u moliya sohasida qanday maqsadlarda qo'llanilishi mumkin?
12. Ma'lumotlar to'plamidagi dispersiya (`np.var()`) va standart og'ish (`np.std()`) ko'rsatkichlari ma'lumotlarning tarqalishini qanday ifodalaydi?

